

> TEXT TO MQL

https://www.linkedin.com/posts/mahesh-ramichetty-160b8121_introducing-text-to-mql-with-langchain-query-activity-7346169718031990784-V0iF

https://www.mongodb.com/company/blog/product-release-announcements/introducing-text-to-mql-langchain-query-mongodb-using-natural-language?utm_campaign=langchain_text_to_mql&utm_source=mongodb&utm_medium=blog&utm_term=langchain

<https://www.mongodb.com/company/blog/technical/natural-language-agents-mongodb-text-mql-langchain>

<https://www.kdnuggets.com/top-7-open-source-ai-coding-models-you-are-missing-out-on>

> TOON Concept

<https://medium.com/data-science-in-your-pocket/toon-bye-bye-json-for-llms-91e4fe521b14>

> Data Extraction

https://www.linkedin.com/posts/sumanth077_turn-complex-documents-into-rag-ready-data-activity-7389605020066832384-aZAo?utm_source=share&utm_medium=member_android&rcm=ACoAAETSLI4BZbC8Kvm_88M-a01ks7fjvI5p1Zk

> Krish Naik Model Training videos

> Yolo Models with Prompts

> Models to be learnt: onnx, microsoft donut, LayoutLM

(<https://medium.com/@shivarama/layoutlmv3-from-zero-to-hero-part-1-85d05818eec4>)

> Google's Speech-to-Retrieval (S2R): A Voice Search Model That Understands Intent Rather Than Words

<https://medium.com/coding-nexus/googles-speech-to-retrieval-s2r-a-voice-search-model-that-understands-intent-rather-than-words-90934f89776f>

> <https://www.youtube.com/watch?v=FepDo-0DrSo>

> <https://www.youtube.com/watch?v=km5-0jyv0JI>

MODEL TRAINING

In deep learning, input data's embeddings are frequently multiplied with weights. This is a fundamental operation within neural networks and particularly prominent in architectures like Transformers.

1) PEFT (Parameter-Efficient Fine-Tuning)

PEFT is an umbrella term for techniques that let you adapt large language models to specific tasks while only training a small fraction of the model's parameters.

The Problem It Solves: Full fine-tuning of a model like GPT-3 or LLaMA requires updating billions of parameters, which demands enormous computational resources and memory. For a 7B parameter model, full fine-tuning might require 100+ GB of GPU memory.

How It Works: Instead of updating all parameters, PEFT methods freeze the original model weights and either:

- Add small trainable modules alongside the frozen model
- Train only a tiny subset of existing parameters
- Learn low-dimensional representations of weight updates

Benefits: You can fine-tune massive models on consumer GPUs, train faster, and easily switch between different task adaptations by swapping out the small trained components.

LORA

LoRA model training involves fine-tuning a large base model by training a small number of new weights, called low-rank matrices, rather than retraining the entire model.

Steps involved in it:

- > preparing a dataset and configuring LoRA parameters,
- > loading a pre-trained base model, applying LoRA to it, and
- > then training the model to fine-tune the low-rank adapters

✓ 1. LoRA Training Pipeline — Step by Step (Simple Version)

Let's assume you want to fine-tune a 7B model (LLaMA/Mistral/etc.).

Step 1 — Load the base model (in 4-bit to save memory)

```
python

model = AutoModelForCausalLM.from_pretrained(
    "mistral-7b",
    quantization_config=BitsAndBytesConfig(load_in_4bit=True)
)
```

Copy code

This loads the original model weights in **4-bit** (compressed format).

Step 2 — Freeze all original model weights

The base model weights are not trainable.

These remain in frozen 4-bit format → requires **very little GPU memory**.

Step 3 — Insert LoRA layers

LoRA adds **small trainable matrices** inside the transformer attention layers.

LoRA mathematically:

```
csharp
```

Copy code

Instead of training weight **W** (big), train two tiny matrices **A** and **B**:

$$W' = W + (A * B)$$

Only A and B are trained.

So instead of updating billions of parameters:

- You train **just a few million**.
- Training is cheap & fast.
- Model quality remains strong.

This happens when you call:

```
python
```

Copy code

```
model = get_peft_model(model, LoraConfig(...))
```

Here the A and B are purely the weights obtained during training. So there weights are added to frozen weight (W). In code these are stored in `adapter_model.safetensors`.

Step 4 — Tokenize your dataset

Convert text → tokens → numeric tensors.

Step 5 — Train only the LoRA parameters

During training:

- Base model (frozen) does **forward pass only**
- LoRA adapters learn the new knowledge

Thus training is:

- Fast
 - Low memory
 - Suitable for 1 GPU
-

Step 6 — Save LoRA adapter weights

Instead of saving 7B parameters, we save only LoRA weights (~30–200 MB):

```
bash

./lora_adapter/
  adapter_model.bin
  adapter_config.json
```

 Copy code



Step 7 — Merge LoRA adapters (optional)

You can combine LoRA + base model into one final model:

```
python

merged = model.merge_and_unload()
merged.save_pretrained("final_model")
```

 Copy code

Now you have a fully fine-tuned model.

✓ 2. What PEFT Does Internally (Simple Explanation)

PEFT = **Parameter Efficient Fine-Tuning**

When you call:

```
python

get_peft_model(model, LoraConfig)
```

 Copy code

PEFT does 3 things:

(A) Finds all target modules

Usually `"q_proj", "k_proj", "v_proj", "o_proj"`
(Attention layers)

(B) Inserts LoRA layers

For each weight matrix W :

```
yaml
Original:
  W (big matrix)  → frozen

LoRA adds:
  A (small)
  B (small)
```

[Copy code](#)

Forward pass becomes:

```
ini
output = W*x + A*(B*x)
```

[Copy code](#)



(C) Marks only LoRA weights as trainable

Use `.requires_grad=True`.

Everything else remains frozen.

QLoRA (Quantized LoRA)

QLoRA combines LoRA with quantization to make fine-tuning even more memory-efficient. You're using QLoRA when a GPU is available, and standard LoRA when only the CPU is detected.

Trade-offs:

- 4-bit: 75% memory reduction, small accuracy drop
- 8-bit: 50% memory reduction, negligible accuracy drop
- 16-bit: 25% memory reduction, essentially no accuracy loss

1) $r=16$: lora rank- Higher $r \rightarrow$ more trainable parameters $\rightarrow$ better quality but more GPU

usage. 2) `target_modules=["q_proj", "k_proj", "v_proj", "o_proj"]`

Apply LoRA to attention layers:

q: query projection

k: key projection

v: value projection

o: output projection

These are the most effective parts to fine-tune

3) lora_dropout: Dropout applied to LoRA layers only. Prevents overfitting.

4) train_model() parameters

a) num_train_epochs- How many times the model will see the entire dataset.

b) learning_rate- Controls how fast LoRA parameters update.

ELK Stack Integration Guide for Python Applications

ELK Stack stands for **Elasticsearch, Logstash, and Kibana** - a powerful trio for centralized logging, search, and visualization:

- **Elasticsearch**: Distributed search and analytics engine that stores logs. In the project Logs are stored in the Elasticsearch index `jett.visa.ocr.api`
- **Logstash**: Data processing pipeline that ingests and transforms logs. In my project we send logs directly from Python to Elasticsearch using the `elasticsearch` Python client so no Logstash involved
- **Kibana**: Visualization interface for exploring and analyzing logs. This picture is shown in the Note section.

This guide demonstrates how to integrate Python applications with ELK for centralized logging and monitoring.

Prerequisites

Required Components

1. **ELK Cluster**: Running Elasticsearch instance (version 8.x in this example)
2. **AWS Account**: For storing sensitive credentials in AWS Systems Manager (SSM) Parameter Store
3. **Python Environment**: Python 3.8+ with required libraries
4. **Network Access**: VPNConnect, Application must reach the Elasticsearch cluster endpoint

This is how an Elastic Cluster looks like. To store logs in ELK, first we have to create cluster in ELK:

```
(myenv) root@DTBLR01LP165:/mnt/d/Projects/elk_test# source myenv/bin/activate
```

```
python - <<'PY'

from elasticsearch import Elasticsearch

from config.jett_config import elastic_config

cfg = elastic_config["logs"]

es = Elasticsearch(

    cfg["url"],
    basic_auth=(cfg["username"], cfg["password"]),

    verify_certs=cfg.get("verify_certs", False),

)

info = es.info()

print(f"Cluster Name: {info['cluster_name']}")

print(f"Cluster UUID: {info['cluster_uuid']}")

print(f"Version: {info['version']['number']}")
```

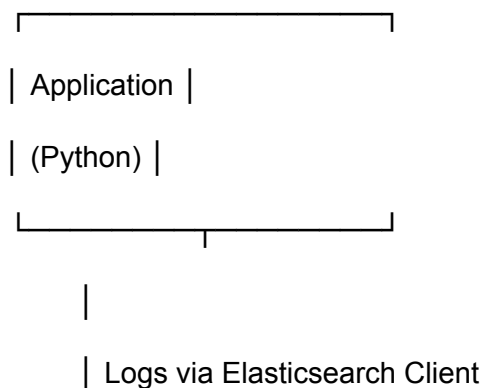
PY

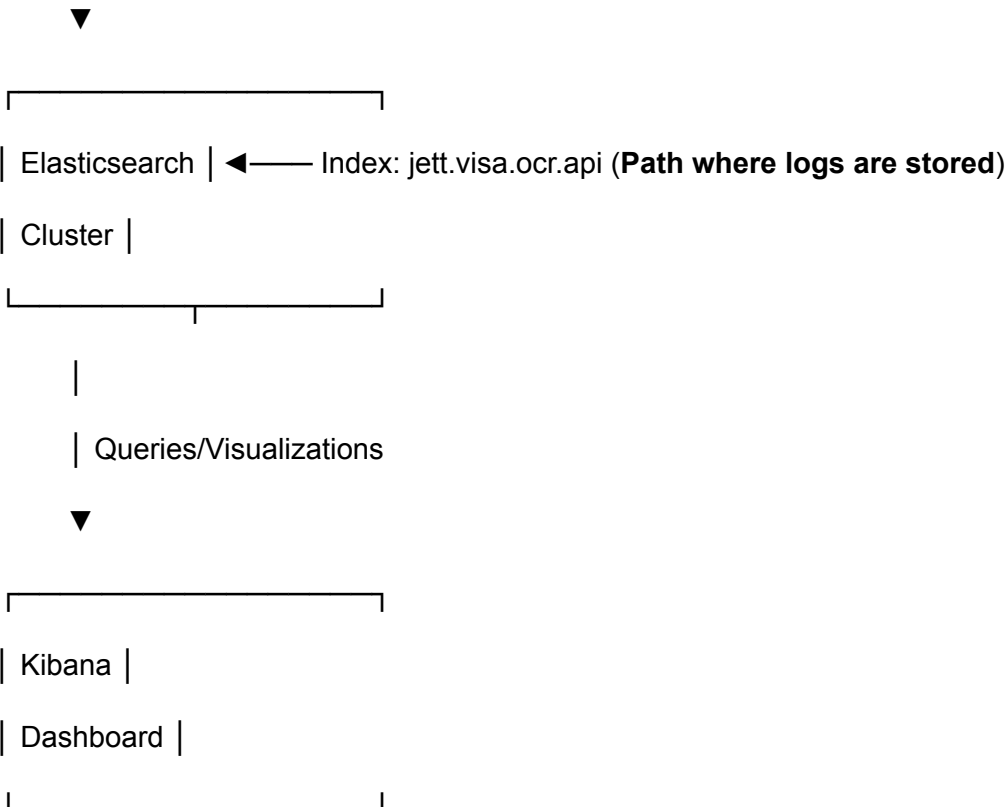
Cluster Name: elasticsearch

Cluster UUID: exgHrPZfThmhpJESmlcVCA

Version: 8.17.0

Architecture





Note:

1) In AWS, SSM refers to AWS Systems Manager (a Service of AWS), a suite of tools for monitoring and managing your infrastructure (It helps to store credentials data in key-value pair). The term "PMS" refers to the Parameter Store capability within AWS SSM, not a separate service, it is a *component* of AWS Systems Manager (SSM) which allows you to store data such as passwords, database strings, and license codes as parameter values.

In this key-value structure storage, the value for Model_key and elk_key and in our project will look something like:

MODEL_KEY: "llm anthropic.claude-3-haiku-20240307-v1:0"

ELK_KEY:

ELK_CONFIG: {

"tracing": "http://10.13.5.230:8200",

"serviceName": "",

"environment": "Dev",

"capturebody": "all",

"captureheaders": "true",

```

"transactionignoreurls":
"/swagger,/swagger/index.html,/swagger/v1/swagger.json,/_vs/browserLink,/_framework/
aspnetcore-browser-refresh.js,/swagger/index.js,/favicon.ico",
"logs": {

"url": "http://10.13.5.230:9200",

"username": "elastic",

"password": "v+uebEQ5=sHaALKCkRd0"

},

"DisableLogsMasking": "ON",

"LogOutput": "console"

}

```

The path in PMS where these credentials are stored are:

MODEL_KEY = f"/v1/kv/data/{ENV}/jett/visa/llm/model"

ELK_KEY = f"/v1/kv/data/{ENV}/jett/visa/apm/elk_config"

Here ENV can be dev or qa or prod anything depending on deployment but it is case sensitive so it **should be initialised in small letters (dev), like**

export ASPNETCORE_ENVIRONMENT=dev

2) We have 2 ELK credentials, one credentials we fetch from AWS and use those to write logs in ELK and with second credentials we log-in into ELK to view logs.

1. Elasticsearch Application Credentials (for Python code)

Purpose: Used by your Python application to write logs to Elasticsearch

Source: Stored in AWS Systems Manager (SSM) Parameter Store

Location in AWS:

- Parameter Path: /v1/kv/data/{ENV}/jett/visa/apm/elk_config
- Where {ENV} = your environment (e.g., dev, qa, prod)

Credentials:

```

URL:      http://10.13.5.230:9200
Username: elastic
Password: v+uebEQ5=sHaALKCkRd0

```

How it's used:

- Fetched by config/jett_config.py from AWS SSM
- Parsed as JSON and stored in elastic_config["logs"]
- Used by utils/log.py to create the Elasticsearch client
- Application writes logs to index: jett.visa.ocr.api

2. Kibana User Credentials (for viewing logs)

Purpose: Used by developers/admins to view and search logs in the Kibana web interface

Source: Managed separately (not in AWS SSM)

Credentials:

URL: <https://dev-apn.musafirbiz.com/app/discover>
Username: JETTApiDeveloper
Password: 9Wr9#zf12gg7?n8n9

How it's used:

1. Open <https://dev-apn.musafirbiz.com/app/discover>
2. Log in with the credentials above
3. Select data view: `jett.visa.ocr.api*`
4. Filter logs using: `source: "visa-chatbot"`

Permissions required:

- Read access to Elasticsearch indices
- Kibana dashboard access
- Ability to create/view data views

Note: This account is separate from the application credentials. It's for human access to Kibana, while the application uses the `elastic` account to write logs.

Why Two Different Credentials?

1. Security best practice: Separate accounts for application automation vs. human access
2. Different permissions: Application needs write access; users need read access
3. Audit trail: Easier to track who/what is accessing the system
4. Flexibility: Can revoke/rotate credentials independently

Summary Table

Aspect	Elasticsearch (Application)	Kibana (User)
Username	<code>elastic</code>	JETTApiDeveloper
Password	<code>v+uebEQ5=sHaALKCkRd0</code>	9Wr9#zf12gg7?n8n9
URL	<code>http://10.13.5.230:9200</code>	https://dev-apn.musafirbiz.com/app/discover
Stored in	AWS SSM Parameter Store	Separate user management
Used by	Python application (<code>utils/log.py</code>)	Developers/Admins (browser)
Purpose	Write logs to Elasticsearch	View/search logs in Kibana
Access Level	Write to indices	Read/view dashboards

Storage flow:

```
Your Python Application
  ↓ (writes via HTTP POST)
http://10.13.5.230:9200
  ↓ (stores in)
Elasticsearch Index: jett.visa.ocr.api
  ↓ (contains)
Your log documents with fields:
- message
- level
- logger
- source: "visa-chatbot"
- @timestamp
- etc.
```

3) To view logs in ELK:

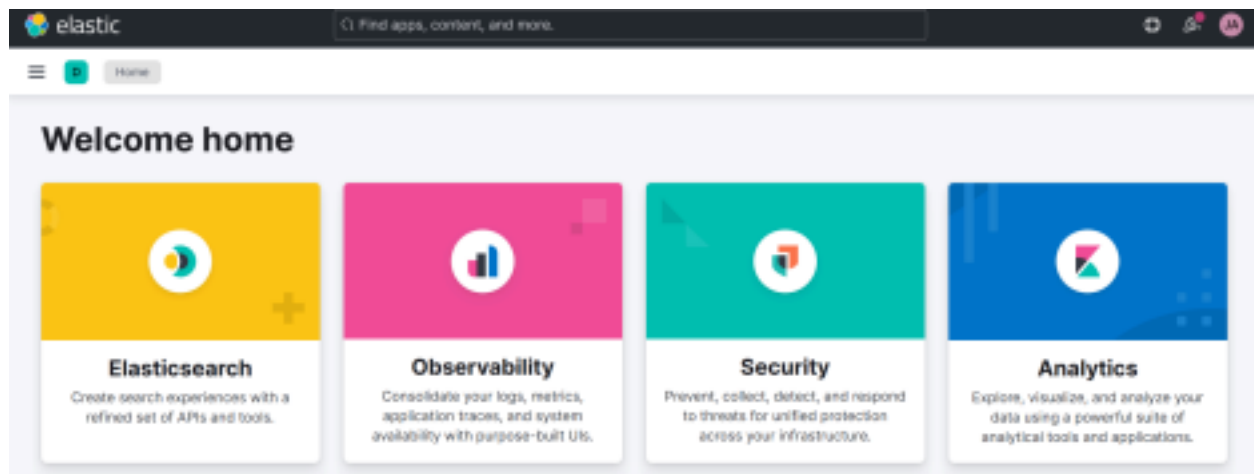
a) Log-in into ELK- Its url, username, password include:

JETTApiDeveloper

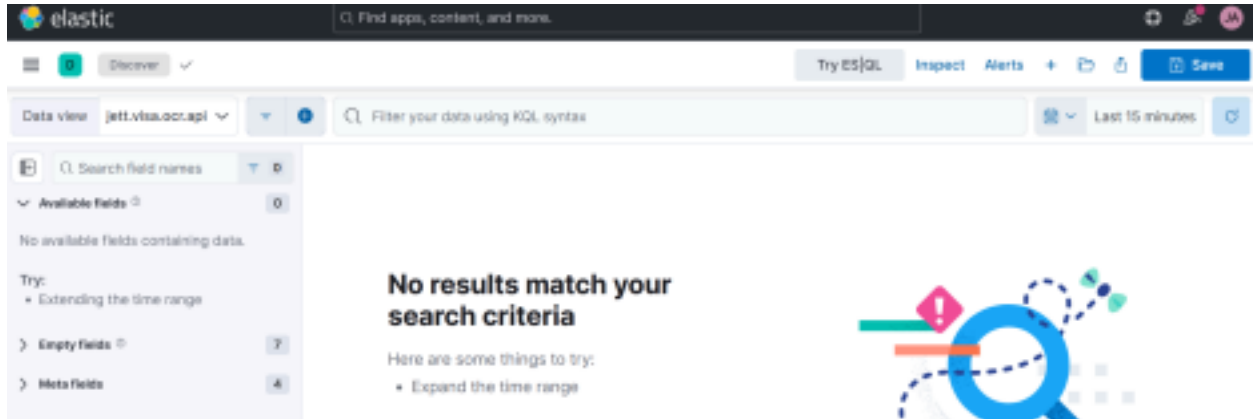
9Wr9#zf12gg7?n8n9

<https://dev-apm.musafirbiz.com/app/discover>

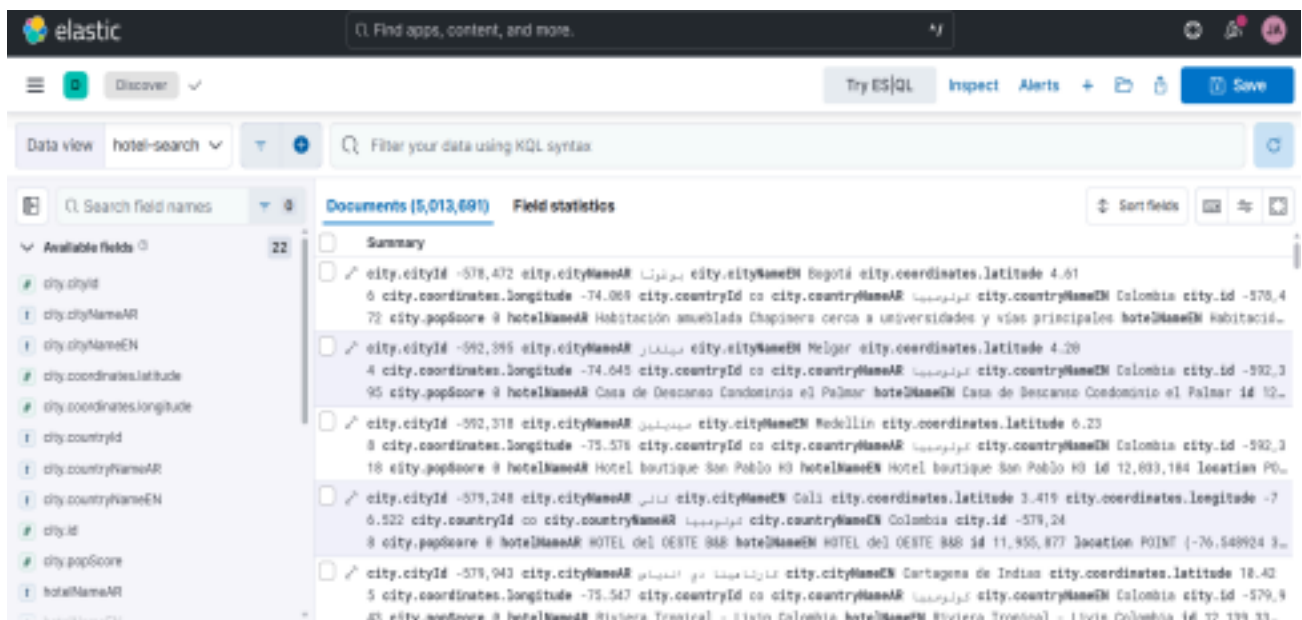
Home Page



b) Go to “discover”



c) Click on “Data View” dropdown and a search field will appear so enter your index_name here (jett.visa.ocr.api) to see the logs as shown below and this is **Kibana page (EL(K))** here K Kibana:



4) ELK Logging Summary

a) config/jett_config.py

> Reads ELK credentials from AWS SSM (parameter /v1/kv/data/{ENV}/jett/visa/apm/elk_config).

> Parses JSON into elastic_config, which holds the Elasticsearch URL, username (elastic), password, index name, and other settings.

> This makes the ELK configuration available to the rest of the app without hardcoding secrets.

b) utils/log.py

> Imports elastic_config from jett_config.py and builds the Elasticsearch client using the

stored credentials.

> Forces compatibility with your cluster (Elasticsearch 8.x) and writes logs to the existing jett.visa.ocr.api index. **Here index means location where logs are stored.**

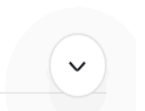
> ElasticSearchHandler formats log records and sends them directly to that index via the REST API.

Together, these files ensure the chatbot logs are routed securely through AWS-configured credentials, written to Elasticsearch, and viewable in Kibana.

Pre-training v/s Fine-tuning

- Pre-training is about building foundational knowledge — like creating a "world model" from scratch.
- Fine-tuning is about specializing that knowledge — like teaching that knowledgeable person to excel at a specific job.

Aspect	Pre-training	Fine-tuning
Goal	Learn general world knowledge and language patterns from vast data.	Adapt a pre-trained model to a specific task or domain .
Stage	First stage — training from scratch.	Later stage — builds on pre-trained models.
Data	Massive, diverse, unlabeled (e.g., internet text).	Smaller, task-specific, often labeled or structured.
Compute	Extremely high (requires hundreds or thousands of GPUs/TPUs, weeks/months).	Much lower (can be done with a few GPUs, hours/days).



Cost	Very expensive (millions of dollars).	Relatively affordable (can be \$100s–\$1000s).
Example	Training GPT-4 or LLaMA on trillions of tokens.	Adapting LLaMA to be a medical chatbot using doctor-patient Q/A data.
Analogy	General education (university).	Specialized job training (medical residency, coding bootcamp).
Outcome	Base model (e.g., GPT-3, BERT).	Specialized model (e.g., GitHub Copilot, ChatGPT-style assistant).

COMPLETE HUGGING FACE CRASHCOURSE

To find or Learn any library of Huggingface, goto 'Docs' section in HF. Here you will get to find ALL libraries of HF like Hub, Transformers etc. <https://huggingface.co/docs>

To Chat anything related to HF, it provides chatbot where you can ask any question related to HF- <https://huggingface.co/chat/conversation/692fc7bac2d3298d242092f2>

To upload or create a new model in HF, goto profile> 'New Model'> create a repository by choosing 'apache 2.0' licence which is open-source from the dropdown> Now new model is created> click on 'Python' to get python code to use this model in the project.

For end-to-end HF implementations of All HF Features refer
huggingface_crash_course.ipynb

FINE-TUNING BERT FOR NLP (NER, Sentiment, QA)

We can train BERT Models using:

- 1) Unsupervised pr-training/ Self-supervised training
- 2) Supervised Training

✓ 1) Unsupervised Pre-training / Self-supervised Training

BERT is first trained on a **large amount of raw text data** (Wikipedia, BooksCorpus).
No labels are required → **self-supervised**.

✓ Two main tasks during BERT pre-training

a) MLM (Masked Language Modeling)

- Randomly mask 15% of tokens
- Model predicts the missing word
Example:
“BERT is a powerful **[MASK]** model.”
Model predicts: “language”.

b) NSP (Next Sentence Prediction)

- Given two sentences A and B
- Model predicts whether B actually comes after A
Used for understanding relationships between sentences.

✓ Why this is “unsupervised”

Because labels (masked tokens, NSP labels) are **generated automatically from the data**, not manually created.

✓ Goal

To make BERT learn:

- grammar
- context
- meaning
- relationships between words and sentences

✓ 2) Supervised Training / Fine-tuning

After pre-training, BERT is trained **again** on a specific downstream task with **human-labeled data**.

Examples of supervised fine-tuning tasks:

- **Classification** → sentiment analysis, spam detection
- **NER** → named entity recognition
- **QA** → question answering
- **Text pair classification** → sentence similarity

✓ Why it is “supervised”

Because now each training example has **labels**.

Example (sentiment analysis):

- “The movie was great!” → *positive*
- “Worst experience ever” → *negative*

The model learns from these labeled examples to perform a particular task.

✓ Goal

To adapt the general BERT knowledge → **task-specific performance**.

Is BERT a generative model?

Short answer: No, BERT is *not* a generative model.

✓ Why BERT is *not* generative

BERT is a bidirectional encoder-only transformer, designed mainly for:

- **understanding text**
- **classification**
- **question answering**

- token-level labeling (NER)
- sentence similarity

It does not generate long, coherent text like GPT, LLaMA, Mistral, etc.

✗ Why BERT can't generate text like GPT

Generative models predict the next token in a sequence (left-to-right).

BERT uses masked language modeling (MLM):

- It predicts only missing (masked) tokens inside a sentence
- Not the next token
- So it cannot autoregressively generate full sentences

Example:

I want to [MASK] a car.

BERT predicts the mask → “buy”.

But it cannot continue:

"I want to buy a car because..."

That's why BERT is not generative.

🧠 BERT is an Encoder-only Model

- Encoder → understands text
- Decoder → generates text
- BERT has only encoder blocks
- GPT has only decoder blocks

So, BERT = understanding model, GPT = generative model.

Where BERT is Still Used Today

1. RAG (Vector Search, RAG Base Models)
 - For generating dense embeddings (e.g., Sentence-Transformers, MiniLM).
 - Stored in FAISS, Pinecone, Milvus for fast similarity search.
 - Used in smaller LLM pipelines (retriever + generator architecture).
2. Enterprise-level NLP Tasks (Fast & Cost-Effective)

Note- Here though BERT is Fast & Cost-Effective, we may have to compromise with Accuracy since it is not a state of the art model like today's LLM's.

- Named Entity Recognition (NER)
 - Sentiment Analysis
 - Classification tasks (spam detection, intent classification)
 - Summarization using lightweight variants (DistilBERT).
4. Multilingual NLP
 - XLM-R (a multilingual BERT version) is still a top choice for 100+ languages.
 - Used for translation and cross-lingual search.
 5. On-Device / Low-Latency Inference
 - For mobile apps and edge devices where GPT/Claude can't run.
 - Quantized DistilBERT/MiniLM models for chatbots and offline NLP tasks.

Model	Main Use-Cases	Why Still Used
BERT / DistilBERT	NER, classification, embeddings	Small, fast, cheap inference
RoBERTa / DeBERTa	Classification, QA, summarization	High accuracy, Kaggle/enterprise use
MPNet / MiniLM	Vector search, semantic retrieval (RAG)	Best for FAISS/Pinecone retrieval
T5 / Flan-T5	Summarization, translation, instruction tasks	Lightweight text-to-text generation
BART / Pegasus	Abstractive summarization	Less resource-hungry than LLMs
XLNet / Electra	Classification, QA (legacy setups)	Still optimized for speed
XLM-R / mT5 / LaBSE	Multilingual NLP, translation, cross-lingual search	100+ language support, enterprise use

Complete BERT Fine-tuning Implementation in BERT_Finetuning.ipynb

LLM Knowledge Distillation

Knowledge Distillation (KD) is a model fine-tuning technique in machine learning where a large, powerful model (Teacher) transfers its “knowledge” to a smaller, faster model (Student) so that the student behaves almost like the teacher but with far fewer parameters and lower computation.

✅ Why do we use Knowledge Distillation?

Because large models are:

- slow to run in production
- costly (GPU/CPU/RAM)
- hard to deploy on devices (mobile, IoT, edge)
- KD helps create compact models while keeping high accuracy.

📦 Where is KD used?

- BERT → DistilBERT
- GPT → TinyGPT variants
- MobileNet, EfficientNet, YOLO
- Compressing LLMs
- On-device ML
- Real-time inference systems

How KD works:

In Knowledge Distillation, the Teacher model transfers its “thinking pattern” to the Student.

Normally, for a classification task, we only give a label like:

Image → Dog

But this label does not tell the model:

- how confident it should be
- whether the image also slightly looks like a cat
- how the teacher distributes probability among all classes

🔥 In KD, the teacher gives much richer information:

Example:

Teacher prediction (**soft probabilities**):

Dog: 0.80

Cat: 0.15

Fox: 0.05

True label says only:

Dog

The teacher says:

"I'm 80% sure it's a dog."

"But it also has some features of a cat (15%)."

"Maybe 5% fox-like."

This additional soft information is the "knowledge" being transferred.

💡 **Why is this more powerful?**

Because the student learns:

- how the teacher differentiates classes
- subtle relationships between classes
- how the teacher generalizes
- the teacher's confidence and uncertainty

This is why **students trained with KD perform better than students trained only on hard labels.**

✅ **"Key Components in KD"**

Knowledge Distillation has three important components:

- 1. Soft Target Loss (KD Loss)**
- 2. Hard Target Loss**
- 3. Temperature (T)**

Let's explain them clearly.

① Soft Target Loss (KD Loss)

This is the main part of KD.

- Student model output → logits
- Teacher model output → softened probabilities
- Student is trained to match teacher's soft probabilities

Mathematically, this is done using KL Divergence between:

- Student predicted prob distribution
- Teacher predicted prob distribution

Why soft targets are useful?

Because they contain information like:

- “second best guess”
- “third best guess”
- level of similarity between classes

This is where most of the teacher's knowledge lies.

② Hard Target Loss (Normal Cross Entropy)

Student also learns from the real labels in the dataset.

Example:

True label: Dog

This prevents the student from drifting too far from ground truth.

Why use both?

- Teacher's predictions guide the student.
- Real labels keep the student correct.

So we combine both.

③ Temperature (T)

Temperature is a very important parameter in KD.

It controls softness of teacher's output probabilities.

Example

Teacher logits → probabilities:

Without temperature (T=1):

Dog: 0.95

Cat: 0.03

Fox: 0.02

With T=5:

Dog: 0.60

Cat: 0.25

Fox: 0.15

Why soften?

- **With softer probabilities, the student can better learn relative similarities.**
- **Hard probabilities (0.95, 0.03, 0.02) hide relationships.**
- **Soft probabilities expose hidden structure (dog looks more like cat than fox!).**

Key point:

Higher T → softer distribution → more teacher knowledge exposed.

I have moondream2 model which for input sign-board image gives output as right-turn, left-turn, go-straight etc based on input image so only this sign board knowledge I want to transfer to mobilenetv-3

Your Goal

You have:

- Teacher model: Moondream2
- Student model: MobileNet-V3

You want MobileNet-V3 to learn only sign-board classification from Moondream2.

Example classes: Right-turn, left-turn, go-straight, u-turn etc

The SIMPLE Idea

STEP 1 — Prepare your images

You already have signboard images like:

`img1.jpg`

`img2.jpg`

`img3.jpg`

`...`

Just a folder of images — that's enough.

No need for labels at this stage.

STEP 2 — Run each image through Moondream2

For each image, ask Moondream2:

“What sign is this?”

Example Moondream2 output:

`Right-turn: 0.90`

Left-turn: 0.05

Go-straight: 0.05

These numbers are called soft probabilities.

What are soft probabilities?

They represent:

- Teacher's confidence
- How similar the classes are
- Knowledge about the image

This is the knowledge you want to transfer.

STEP 3 — Save these probabilities

For every image, save:

img1.jpg, [0.90, 0.05, 0.05]

img2.jpg, [0.02, 0.96, 0.02]

img3.jpg, [0.10, 0.10, 0.80]

This becomes your *training target* for MobileNet-V3.

STEP 4 — Train MobileNet-V3

Now you train MobileNet-V3 using:

👍 Input → the image- with which you got output from Teacher model

👍 Target → the teacher's probabilities- soft probabilities

MobileNet-V3 tries to learn:

- “If an image looks like this → Moondream2 says right turn 90%.”

So MobileNet learns to behave like Moondream2 for signboard images.

STEP 5 — Student learns 2 things

During training, MobileNet-V3 learns from:

1 Teacher's soft outputs (main knowledge)

These are Moondream2's probabilities.

2 True labels (optional)

If you have actual labels like:

`img1 → right-turn`

you can include them too.

But not required.

STEP 6 — After training

MobileNet-V3 becomes:

- Tiny
- Fast
- Accurate
- Knows only traffic sign classification
- Behaves like Moondream2 (for this task)

Refer [LLM Knowledge Distillation.ipynb](#)

QUANTIZATION

PART-1

✓ What is Quantization? (Simple Explanation)

Quantization is the process of converting a model's weights and biases from high-precision numbers (like float32) to lower-precision numbers (like int8, float16, or even int4).

✓ Why Do We Quantize a Model?

To make models:

- Smaller in size (4× smaller when going from 32-bit → 8-bit)
- Faster to run (lower-precision operations are cheaper)
- Cheaper to deploy (less RAM, less compute)
- More suitable for edge devices like phones, microcontrollers, Jetson Nano, - Raspberry Pi, etc.

✓ What Parts of a Model Can Be Quantized?

Different model types have different quantizable components.

Model Type	Quantizable Weights/ Which part of the model needs to be Quantized
1) ANN	Weights between hidden layers
2) CNN	Convolution filters
3) RNN / LSTM	Recurrent matrices (e.g., hidden-state transition matrices)
4) Transformer / LLM	Query, Key, Value matrices and FFN (Feed-Forward Network) weights

1) ANN Quantizable Part

In a simple neural network:

Input → Hidden Layer 1 → Hidden Layer 2 → Output

Each neuron in Layer 1 connects to every neuron in Layer 2.

For example:

Layer 1 has 128 neurons

Layer 2 has 64 neurons

Then it has a weight matrix of shape: 128×64

✓ **Why can these weights be quantized?**

Because this weight matrix is usually very large and contains millions of numbers.

Quantization converts:

float32 → int8 (4× compression)

float32 → int4 (8× compression)

Since ANN dense layers do simple multiplications, they tolerate precision loss well.

✓ **Example:**

Original weight: 0.892345 (float32)

Quantized weight (int8 mapped): 89

The model still works nearly the same.

2) CNN Quantizable Part: Convolution Filters

In CNNs, convolution layers use small grids of numbers called filters or kernels.

Example:

3×3 kernels

5×5 kernels

7×7 kernels

For a 3-channel RGB image, a 3×3 kernel looks like: $3 \times 3 \times 3 = 27$ weights. So these weights values needs to be quantized. A CNN may have hundreds of such kernels.

✓ 3) Quantizable Parts in Transformers / LLMs

Transformers have two major components:

A) Self-Attention- Uses Query (Q), Key (K), Value (V) matrices

B) Feed-Forward Network (FFN)- Two big dense layers inside each transformer block.

★ A) Query (Q), Key (K), Value (V) Matrices

Every transformer block computes attention:

$$\text{Attention} = \text{softmax}(Q \times K^T) \times V$$

To get Q, K, and V, the model multiplies input 'x' with 3 weight matrices:

$$Q = x \times W_q$$

$$K = x \times W_k$$

$$V = x \times W_v$$

✓ What do these matrices do?

Query (Q): What am I looking for?

Key (K): What information do I have?

Value (V): The actual content to pass along

These matrices determine which words attend to which words (crucial for LLMs).

✓ Why can we quantize Q, K, V?

Because each matrix is huge. For a 13B parameter LLM 40–50% of its parameters belong to QKV matrices. Quantizing them gives massive size reduction. Transformers are known to tolerate int8 / int4 quantization extremely well.

★ B) FFN (Feed-Forward Network) Weights

Inside each transformer layer, after attention, there is a feed-forward network:

$x \rightarrow \text{Dense} \rightarrow \text{Activation} \rightarrow \text{Dense} \rightarrow \text{Output}$

These dense layers are often 4× larger than the hidden size.

✓ Why quantize FFN weights?

Because FFN layers have the largest number of parameters in the entire LLM.

Example (simplified):

If a transformer has 10 million parameters, almost 7–8 million are in the FFN layers.

These weights also form large dense matrices → perfect for low precision.

Note:

- 1) “Biases can also be quantized (though optional)”
- 2) “Activations are also quantized during inference (for speed) or training (in QAT)”
 - ♦ Activations = outputs of a layer

For example: $\text{ReLU}(xW + b)$ → this output is activation so this output can be quantized.

In QAT (Quantization-Aware Training)

- Both weights and activations are quantized during training.
- The model learns to handle low precision.
- Purpose → best accuracy after quantization.

3) IN QUANTIZATION, PRECISION MEANS THE NUMBER OF VALUES PRESENT AFTER THE DECIMALS POINT. HIGHER THE NUMBER OF VALUES AFTER DECIMAL REFERS TO HIGH PRECISION. Refer below examples

Simple Analogy:

Imagine you're storing prices of products:

In full detail: ₹1299.956 (float16)

After rounding: ≈1300 (int8 style)

$$1300 - 1299.956 = 0.044$$

You lose tiny details, but the core information remains. This is exactly what quantization does to model weights, biases and activations.

1. Mathematical Value / PI / Float Examples

3.14159265- High precision (float32)

3.14- Low precision (float16)

3- Lowest precision (INT 8)

2. Currency Example (e.g. Dollar / ₹ Rupee)

₹87.5793- High precision (float32-like)

₹87.573- Medium precision (float16-like)

₹87- Low precision (int8/int4-like)

Why Quantization Matters?

1. Reduce Model Size

A model stores millions or billions of weights (numbers). How much space these weights take depends on their datatype.

♦ For FP32 weights (normal / float32)

- Each weight = 4 bytes
- This is the standard precision during training.
- Example: 1 billion weights → 4 GB just for weights.

♦ For INT8 weights (quantized)

- Each weight = 1 byte
- This is a lower-precision version.

★ So INT8 gives ~4× smaller model size

Because:

FP32 → 4 bytes

INT8 → 1 byte

Model size reduces 4 times

Example:

Model precision	Size
-----------------	------

FP32	4 GB
------	------

INT8

1 GB

So quantization shrinks the model drastically.

2. Lower Memory (RAM/VRAM) Usage

When the model becomes smaller, it also takes **less GPU VRAM** or **CPU RAM** to load.

✓ Quantized models consume less memory

Why? Because storing 1-byte numbers (INT8) is much cheaper than 4-byte numbers (FP32).

✓ Meaning of the bullet points

“Load large models on small GPUs (e.g., LLaMA 7B on a 6GB GPU)”

Normally:

- A 7B FP32 model takes ~28 GB VRAM
(7B weights × 4 bytes)

But after INT8 quantization:

- It becomes ~7 GB
(7B weights × 1 byte)

This allows small GPUs like:

- NVIDIA 3060 6GB
- T4 8GB
- RTX 2060 6GB

to load models that earlier required 24–30 GB VRAM.

PART- 2

1. What is RAM?

RAM = Main memory of your computer (system memory).

✓ **RAM stores:**

- OS processes
- Program data
- Data needed during execution
- Neural network model (only if running on CPU)

✓ **Examples:**

- Opening Chrome → tabs and browser data remain in RAM
- Running a Python script on CPU → RAM used
- Running a machine learning model on CPU → model loaded into RAM

📌 Think of RAM as a temporary workspace for the CPU.

■ **2. What is CPU (Central Processing Unit)?**

CPU = The general-purpose "brain" of your computer.

✓ **Role:**

- Executes sequential operations
- Runs programs step-by-step
- Good for logic, branching, small tasks

✓ **Examples:**

- Running Python scripts
- Browsing the web
- Running normal applications

📌 CPU is powerful but not specialized for ML matrix operations.

■ **3. What is VRAM? (Video RAM)**

VRAM = Dedicated memory present on your GPU.

(Found in NVIDIA / AMD graphics cards)

✓ **VRAM stores:**

- Model weights
- Activations (during forward/backward pass)
- Gradients (during training)

- Images / Tensors

✓ Examples:

- Running LLaMA or Stable Diffusion on GPU → weights stored in VRAM
- If VRAM is not enough → model crashes with OOM (Out Of Memory) error

📌 VRAM is only for GPU, while RAM is for CPU.

📌 VRAM is usually smaller (in laptops) but much faster than RAM.

■ 4. What is GPU (Graphics Processing Unit)?

GPU = Highly parallel processor optimized for matrix/tensor operations.

✓ Role:

- Executes huge matrix multiplications
- Highly parallel (thousands of cores)
- Ideal for training + inference of ML models

✓ Examples:

- Running LLaMA, Stable Diffusion, GPT-like models
- Playing modern video games
- Training deep learning models

Feature	RAM	VRAM
Used by	CPU	GPU
Speed	Slower than VRAM	Extremely fast (optimized for tensors)
Application	Web, apps, CPU training	Deep learning inference/training on GPU
Capacity	8–64 GB typical	2–48 GB (depending on GPU)
Model hosted?	On CPU	On GPU
Quantization helps?	✓ Yes (smaller RAM needed)	✓ Yes (smaller VRAM needed)

Bit & Byte Notes: 8 bits = 1 byte & float32 = 32 bits = 4 bytes per number

Data Type	Meaning	Size in Memory	Precision	Example Value	Precision Meaning	Usage in DL
float64	64-bit floating point (FP64) <i>(optional)</i>	8 bytes	Very High	3.141592653589793	~15–16 digits (scientific)	Scientific computing, double precision
float32	32-bit floating point (FP32)	4 bytes	High	3.14159265	~7 decimal digits	Training & baseline
float16	16-bit floating point (FP16)	2 bytes	Medium	3.14	~3–4 decimal digits	Mixed-precision training (NVIDIA AMP)
bfloat16	Brain float 16 (Google TPU)	2 bytes	Medium	3.14 (fewer mantissa bits)	~3–4 decimal digits	Training on TPUs
fp8	8-bit float (E4M3 or E5M2)	1 byte	Low-Medium	~2.75	~2–3 digits	NVIDIA Hopper GPUs, inference/training
nf4	Normal Float 4 (log-based float)	0.5 bytes	Very Low	0.5, 1.0, 2.0	Very coarse float	bitsandbytes, GPTQ, AWQ
int8	8-bit integer	1 byte	Low	–128 to +127	Exact integer (no decimals)	Quantized inference
int4	4-bit integer	0.5 bytes	Very Low	–8 to +7	Very coarse	GPTQ, AWQ, extreme quantization
int32	32-bit integer	4 bytes	Exact	123456	Integer precision	Indexing, padding, tensor ops

bool	Boolean (True/False)	1 bit	None	True, False	Binary only	Attention masks, binary ops
-------------	-------------------------	-------	------	-------------	----------------	--------------------------------

Data Type	Memory Required
float32	$100\text{M} \times 4\text{B} = \mathbf{400\text{MB}}$
float16	$100\text{M} \times 2\text{B} = \mathbf{200\text{MB}}$
int8	$100\text{M} \times 1\text{B} = \mathbf{100\text{MB}}$
int4	$100\text{M} \times 0.5\text{B} = \mathbf{50\text{MB}}$

Deep learning engineers intentionally switch data types based on use-case:

- **FP32 → Full training**
- **FP16/BF16 → Mixed precision training**
- **INT8/INT4 → Inference**

PART- 3

What is Precision in Deep Learning?

It refers to the number of values after the decimal point.

Technical View

- **float32 = 32 bits**
 - Can store very **fine decimal values** (like 0.00000001)
- **int8 = 8 bits**
 - Can store only **whole numbers** from **−128 to 127**
 - No decimal → **Low precision**

- So,
- float32 is high precision
 - int8 is low precision

Why Does Precision Matter?

- During **training**, we need **high precision** to learn small gradients and improve accuracy.
- During **inference**, we can use **low precision** (e.g., int8), because we only need fast predictions — even if they are **slightly less accurate**.

Quantization Formats Overview

Format	Bit Width	Used For	Precision Level	Speed	Accuracy
FP32	32-bit	Full precision training	◇ High (~7 decimal digits)	Slow	☑ Best
FP16	16-bit	Mixed precision training	◇ Medium (~3–4 decimal digits)	Faster	△ Slight drop
INT8	8-bit	Most common for quantization	○ Low (no decimals, –128 to +127)	Fastest	▽ Small accuracy loss
INT4 / NF4	4-bit	Research, LLM compressi	▽ Very Low (coarse	Ultra-Fast	✖ Moderate

		on (bnb/GPT Q)	discrete/ float values)		accu racy loss
FP8	8-bit	NVIDIA Transformer inference (H100)	○ Low-Me dium (~2–3 decimal digits)	Fast	△ Sligh t drop

Note:

Precision ≠ Accuracy — What Does It Mean?

◇ Precision- "How finely or in how much detail a number is represented or stored."

◇ Accuracy- "How close is the prediction/output to the correct/true value."

PART- 4

Formula of Quantization and Dequantization

Important Note- This formula converts weights into Quantized values.

Quantization (Float → Int)

$$x_q = \text{round} \left(\frac{x}{\text{scale}} \right) + \text{zero_point}$$

Where:

- **x** = original float value
- **x_q (x_q)** = quantized integer (e.g., int8)
- **scale** = how much range one int step covers
- **zero_point** = which integer represents 0.0 in float

- **round()** = nearest integer

Dequantization (Int → Float)

$$x = (x_q - \text{zero_point}) \times \text{scale}$$

This is used during inference to convert quantized weights back to usable float values inside the neural network.

What is the Goal of Quantization?

To store **real numbers** (like float32: 1.23, -0.85, 2.77...) in **less memory** using small integers such as **-128 to +127** or **0 to 255**.

Types Of Quantization:

1) PTQ – Post-Training Quantization

The model is already trained in float32. We convert it to int8 without retraining during quantization.

2) QAT = Quantization-Aware Training

The model is trained with quantization in mind — it learns to handle quantization during training itself.

PTQ (Post-Training Quantization):

- Train with a perfect bat, play with a cheap bat later
- You train the player with a high-quality bat (float32 weights).
- But in the match, you suddenly give him a plastic bat (int8 weights).
- He might still perform okay, but maybe not perfect- he never practiced with this bat!

QAT (Quantization-Aware Training):

- Train and play with the same cheap bat
- You train the player from day one with the exact bat (int8) he'll use in matches.
- So he learns how to handle its limitations.
- Outcome: more consistent and reliable performance in the real match.

LLM Fine-Tuning

1) Parameter based Fine-tuning

- a) LORA
- b) QLORA

2) Data-level Fine-tuning

- a) Instructional Fine-tuning
- b) Non-instructional Fine-tuning

a) Instructional Fine-tuning

Instructional fine-tuning is the process of training a pretrained language model on instruction-response pairs (means training the model with both question and answer) so that the model learns how to follow user commands.

Dataset:

Follows **Instruction + Response** format.

Use case:

Chatbots, Q&A systems, tutors, coding assistants, customer support, etc.

Goal:

Teach the model to **follow human instructions** — turning it into a chatbot or AI assistant.

Output style:

Direct, helpful, and structured answers — often with reasoning or explanations.

Example (IFT dataset row)

Instruction:

“Explain what mitochondria does in one sentence.”

Response:

“Mitochondria generate energy for the cell through respiration.”

Instruction Fine-Tuning (SFT) – Public Conversational Data

- **Reddit**
 - r/AskReddit
 - r/explainlikeimfive

- **Quora**
 - Human Q&A style data
- **StackOverflow / StackExchange**
 - Technical question–answer patterns
- **GitHub Issues / Discussions**
 - Coding and troubleshooting threads

Training Data Sample

Instruction: Explain what mitochondria does in one sentence.

Response: Mitochondria generate energy for the cell through cellular respiration.

b) Non-instructional Fine-tuning

Non-instructional fine-tuning is a training method where a pretrained language model is further trained on raw text only like PDF etc, without instructions, questions, or answers. Non-instructional fine-tuning = continuing language-model training on plain domain text to adapt the model to a specific domain or writing style.

It is also called Language Modeling Fine-Tuning

What the Training Data Looks Like-

- ✓ Plain text
- ✓ PDFs converted to text
- ✓ Articles, reports, logs, contracts
- ✗ No prompts
- ✗ No Q&A pairs
- ✗ No chat format

How Training Works: Input text → Model → Predict next word/token

Use Case: Make LLM a domain expert

After fine-tuning:

- > A medical LLM writes like a doctor
- > A legal LLM writes like a lawyer
- > A finance LLM understands balance sheets better

⚠ But it may still fail at following instructions like: “Explain this to a 5-year-old”

Example (Normal Fine-Tuning Dataset Row)

Sunny Savita here AI meets unstoppable passion. Turning code, creativity, and courage into a legacy. Teaching thousands to master AI, one video at a time. Not just a creator, a mentor, a visionary, a job-maker. Don't chase opportunities — build them.

Input (model ko diya gaya text)	Output (model se expected next token)
"Sunny"	"Savita"
"Sunny Savita"	"_"
"Sunny Savita -"	"where"
"Sunny Savita - where"	"AI"
"Sunny Savita - where AI"	"meets"
"Sunny Savita - where AI meets"	"unstoppable"

Combo Strategy (Common in Industry)- Instructional Fine-Tuning + Non-Instructional Fine-Tuning

Step 1: Non-Instructional Fine-Tuning

→ Adapt LLaMA (or any base model) to your domain

(e.g., pharmaceutical documents, legal text, financial reports)

Step 2: Instruction Fine-Tuning

→ Train the model on user question–answer or task instruction data

(Optional) Step 3: RLHF

→ Refine the model using real user feedback and preference ranking

Note- ChatGPT is trained in a similar way using these 3 steps.

Preference Alignment / Preference Training

Preference alignment is the process of training a model to respond in ways humans prefer, not just in ways that are technically correct.

User question: “How do I lose weight fast?”

Model answer: “It’s simple, just stop eating so much.”

So here even though it answers the question, it is not aligned with human expectations.

Preference-Aligned Model answer

“Safe weight loss generally happens gradually.

You can start with balanced meals, regular movement, adequate sleep, and hydration.

If you have medical conditions, it’s best to get a personalized plan from a professional.

I can help you with simple steps if you want!”

How preference alignment is done in practice:

RLHF (Reinforcement Learning from Human Feedback)

DPO (Direct Preference Optimization)

RLAIF (AI-generated feedback instead of humans)

Dataset Structures of each type of Fine-tuning

1 Unsupervised Pretraining / Non-Instruction Fine-Tuning

 **What the data looks like:** Just raw text, no instructions, no Q&A.

Example paragraph: Metformin is one of the most widely prescribed oral antihypertensive agents...

 **How training works:** The model learns by next-token prediction.

Input (context so far)	Output (next token)
"Metformin is one of the most widely prescribed"	"oral"
"Metformin is one of the most widely prescribed oral"	"antihypertensive"
"Metformin is one of the most widely prescribed oral antihypertensive"	"agents."

2 Instruction Fine-Tuning (IFT)

 **What the data looks like:** Text is structured as Instruction → Input → Response

Common formats: Instruction / Input / Output

♦ Example 1 (Summarization)

```
{  
  "instruction": "Summarize the following paragraph.",  
  "input": "Clinical trials have demonstrated...",  
  "output": "Atorvastatin and Ezetimibe combination lowers LDL-C..."  
}
```

♦ Example 2 (Emotional / Conversational)

```
{  
  "context": "I'm feeling worthless and can't sleep.",  
  "response": "You are not alone, therapy can help you regain balance."  
}
```

3 Preference Training (Reward / Alignment Training)

📌 **What the data looks like:** Instead of “correct answer”, you give good vs bad answers.

```
{  
  "prompt": "Explain mechanism of Metformin",  
  "chosen": "Metformin activates AMPK...",  
  "rejected": "Metformin just stops sugar absorption."  
}
```

📌 **How training works:** “For this prompt, this answer is preferred over that one”

Types of Preference Training include- RLHF, DPO etc. Here DPO is most advanced.

Direct Preference Optimization (DPO)- A Type of Preference Training

Direct Preference Optimization (DPO) is a simpler, more efficient method for fine-tuning large language models (LLMs) to align with human preferences, bypassing the complex reinforcement learning (RL) steps used in traditional Reinforcement Learning from Human Feedback (RLHF), by directly optimizing the model using pairs of preferred/rejected responses with a simple loss function.

How DPO Works

Data: Uses preference data: a prompt, a preferred response, and a dispreferred response.

Goal: Adjusts the LLM's policy (its response generation) to favor preferred outputs over rejected ones.

Mechanism: Implements this adjustment as a simple classification problem using a binary cross-entropy loss, making it computationally lighter than RLHF.

In DPO, the task of aligning a model to human preferences is mathematically reframed as a binary classification problem.

Binary Classification: This is a type of machine learning problem where the goal is to predict one of two possible outcomes, typically labeled as 0 or 1 (e.g., spam or not spam, good response or bad response).

DPO Context: For any given prompt, the model is presented with two potential responses from the preference dataset: a "preferred" (chosen) response and a "non-preferred" (rejected) response. DPO treats the act of selecting the "preferred" response over the "non-preferred" one as the "correct" classification (label 1), and selecting the other as the "incorrect" one (label 0).

Binary Cross-Entropy (BCE) Loss: This specific loss function measures the difference between the model's predicted probability for an outcome and the actual binary label (0 or 1). By using BCE loss, the model is penalized when it assigns a high probability to the rejected response or a low probability to the preferred one. Minimizing this loss encourages the model to assign a higher probability to the 'correct' class (the preferred response).

Simplicity: This approach is much simpler than traditional Reinforcement Learning from Human Feedback (RLHF), which requires training a separate reward model and managing a complex online sampling process using algorithms like PPO. DPO bypasses all this, allowing the model to be trained with standard supervised learning techniques and gradient descent, similar to how basic classification models are trained.

Key Idea: The model learns to assign higher probability to the good response and lower probability to the bad response for the same prompt.

The DPO model learns this through the process of minimizing the binary cross-entropy loss function during training via gradient-based optimization (e.g., stochastic gradient descent).

The Loss Function's Role: The DPO loss function is specifically designed to maximize the log probability ratio of the preferred response relative to the rejected response, while keeping the model from drifting too far from its original, fixed "reference" model.

The Learning Mechanism: For each training example (a prompt with a chosen response and a rejected response):

- The model calculates the probability of generating both the preferred and non-preferred responses.
- The loss function calculates a "penalty" based on these probabilities. A high loss occurs if the probability of the preferred response is low, or if the probability of the rejected response is high
- During training, the model's internal parameters (weights) are adjusted (via backpropagation) to reduce this loss.
- These adjustments effectively nudge the model to increase the likelihood of generating the preferred response and decrease the likelihood of generating the non-preferred response for that specific prompt.

1 The DPO Loss (Big Picture)

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y^+, y^-)} \left[\log \sigma \left(\beta \left(\log \frac{\pi_{\theta}(y^+|x)}{\pi_{\text{ref}}(y^+|x)} - \log \frac{\pi_{\theta}(y^-|x)}{\pi_{\text{ref}}(y^-|x)} \right) \right) \right]$$

This formula "Make the model assign higher probability to the preferred answer than the rejected one — relative to a reference model."

Here,

x- Prompt
y⁺- Preferred (chosen) response
y⁻- Rejected response
π_θ- Current model being trained
π_{ref}- Frozen reference model (SFT model)
β- Preference sharpness / strength
σ- Sigmoid function

Here,

1)

$$\log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$$

The above part of the formula tells “How much more does my current model like this answer compared to the reference model?”

- Positive → model prefers it **more than reference**
- Negative → model prefers it **less than reference**

2) (chosen improvement)–(rejected improvement)

It Speaks:

[How much model improved on chosen]

[How much model improved on rejected]

Desired outcome: This difference should be positive

Means: Model prefers chosen more than rejected

3) Why the Sigmoid σ(·): Sigmoid converts any real number into (0, 1)- Turns preference difference into a probability-like score

Large positive ≈ 1 (very good preference)

0- 0.5 (neutral)

Negative ≈ 0 (bad preference)

4) Why the Negative Sign (-)?: We minimize loss during training.

–log(good thing)

So minimizing loss → maximizing preference correctness

“We want chosen > rejected as strongly as possible”

5) **Role of β (Beta Scaling Factor):** β controls how strongly preferences are enforced.

High β - Strongly push chosen $\gg$ rejected

Low β - Soft, conservative preference

Typical- 0.1 – 0.5

Stack Of LORA

Step1- I trained a model using non-instructional fine-tuning with LoRA. On top of this

Step2- I then trained on top of a non-instructional model using instructional fine-tuning with another LoRA.

Step3- After that, I further trained on top of this instructional model using preference-based training with yet another LoRA.

This condition is called "Stack of Lora" which is not a good practice to perform. In such cases we use `PeftModel.from_pretrained()` and `merge_and_unload()` instead of creating a brand new LoRA adapter again before `get_peft_model()` in the code.

Stage	What You Should Do	Wrong Way (you did)
Non-Instruction	Base + LoRA	✓ correct
Instruction	Base + merge(stage1 LoRA) + NEW LoRA	✗ "LoRA on LoRA"
Preference	Base + merge(stage2 LoRA) + NEW LoRA	✗ "LoRA on LoRA on LoRA"

`get_peft_model()`: This function is used to create a brand new LoRA adapter and attach it to a model, usually for a new training phase. If you apply `get_peft_model()` to a model that already has an active LoRA adapter, you'd be putting a LoRA on top of another LoRA. Your notebook calls this 'Stack of LoRA', which is often not ideal.

`PeftModel.from_pretrained()`: This is used to load an already existing and trained LoRA adapter and apply it to a base model. This is great for inference or further training of that specific adapter.

Where **merge_and_unload()** comes in:

- When you're doing sequential fine-tuning (like your three steps: Non-Instructional LoRA -> Instructional LoRA -> Preference LoRA), you want each step to build cleanly on the previous one.
- You start with a `base_model`.
- You apply a LoRA for non-instructional tuning. This creates a `PeftModel` with an active LoRA adapter.
- Now you want to do instructional tuning. If you just used `get_peft_model()` again on the non-instructional `PeftModel`, you'd have a 'LoRA on LoRA'.

So `merge_and_unload()` solves this! It takes the non-instructional LoRA weights and literally merges them into the `base_model`'s weights. The `PeftModel` then becomes a regular `AutoModelForCausalLM` again, but one that has now learned from the non-instructional tuning. It's like the LoRA's lessons are now permanently written into the model itself.

After `merge_and_unload()`, you have a new, instruction-tuned base model. You can then apply a fresh LoRA adapter for your preference training using `get_peft_model()`.

Refer

- 1) `non_Instruction_pretrain_llm_finetuning_on_domain_specific_data.ipynb`
- 2) `Instruction_finetuning_on_domain_specific_dataset.ipynb`
- 3) `Preference_Aligned_Training_DPO_final.ipynb`

LLAMA FACTORY

LLaMA Factory is an all-in-one open source easy-to-use and efficient platform for training and fine-tuning large language models. With LLaMA Factory, you can fine-tune hundreds of pre-trained models locally without writing any code.

LLaMA Factory is a UI and CLI based framework that lets you fine-tune LLMs without writing code.

UI (LlamaBoard / WebUI): A beginner-friendly web interface where you can select models, upload datasets, set parameters, and run training, export, or chat — all through a simple GUI.

CLI: If you prefer scripting, automation, or advanced control, the command-line interface lets you run training, export models, start chat, or serve APIs directly from the terminal.

Dataset format supported by llama-factory

Alpaca Format (Instruction Finetuning)

```
[  
  
  {  
  
    "instruction": "What is preference alignment?",  
  
    "input": "",  
  
    "output": "Alignment is making LLM follow human preferences..."  
  
  }  
]
```

Columns:

- instruction
- input
- output

ShareGPT Format (Chat finetuning)

```
[  
  
  {  
  
    "conversations": [  
  
      {"from": "user", "value": "Hello"},  
  
      {"from": "assistant", "value": "Hi, how can I help you?"}  
  
    ]  
  
  }  
]
```

DPO Format (Preference Training)

```
[  
  
  {  
  
    "prompt": "Explain RLHF.",  
  
    "chosen": "RLHF aligns model with human feedback.",  
  
    "rejected": "I don't know about RLHF."  
  
  }  
  
]
```

Step-by-Step Finetuning (Web UI) in LLAMA Factory

- 1) Base Model Select
- 2) Dataset Load
- 3) Training Configuration
- 4) GPU Optimization
- 5) Start Training
- 6) Evaluation
- 7) Chat With Your Model
- 8) Export Model

CLI Training (YAML-Based)

Command	Purpose / Description
llamafactory-cli train config.yaml	Run a training/fine-tuning session (LoRA, QLoRA)
llamafactory-cli chat config.yaml	Launch CLI chat after training/export
llamafactory-cli eval config.yaml	Evaluate model (perplexity/custom eval)
llamafactory-cli export config.yaml	Merge adapters and export final model
llamafactory-cli api config_api.yaml	Start an API server endpoint (OpenAI-style)
llamafactory-cli webui	Launch graphical interface (training, eval, chat, export)

llamafactory-cli webchat

Launch web-based chat UI

llamafactory-cli version

Show installed version

